

Projeto Final: Desenvolvimento do Jogo Super Trunfo

Disciplina: Algoritmos e Programação I (Python)

Este documento apresenta as diretrizes, requisitos e restrições para o projeto final da disciplina. O objetivo é desenvolver uma versão funcional do jogo de cartas **Super Trunfo**, utilizando exclusivamente os conceitos de lógica de programação e estruturas de dados básicas abordadas em aula.

1. Objetivo do Projeto

O aluno deverá implementar o motor de um jogo Super Trunfo, permitindo que um usuário humano jogue contra o computador (Single Player) ou contra outro usuário (Dual Player). O foco reside na manipulação de **listas de listas** para representar o deck e as mãos dos jogadores.

2. Regras do Jogo

- O baralho é composto por diversas cartas, cada uma com um nome e um conjunto de atributos numéricos (ex: Velocidade, Peso, Potência).
- As cartas são distribuídas aleatoriamente entre os dois jogadores.
- A cada rodada, o jogador da vez escolhe um dos atributos de sua carta do topo.
- O valor desse atributo é comparado com o valor do mesmo atributo na carta do topo do adversário.
- Quem tiver o maior valor vence a rodada e leva ambas as cartas para o final do seu monte.
- O jogo termina quando um jogador possuir todas as cartas do baralho.

3. Requisitos de Implementação e Restrições (Mandatário)

Para garantir que o projeto avalie os conhecimentos específicos de Algoritmos I, as seguintes restrições devem ser obedecidas:

Recurso	Status	Observação
Dicionários (dict)	PROIBIDO	Toda a informação deve ser armazenada em listas.
Classes (OOP)	PROIBIDO	Não é permitido o uso de programação orientada a objetos.

Recurso	Status	Observação
Listas (list)	OBRIGATÓRIO	Uso de listas aninhadas (matrizes) para as cartas.
Biblioteca random	OBRIGATÓRIO	Para embaralhar e distribuir as cartas no início.

4. Estrutura de Dados das Cartas

Cada carta deve ser representada por uma lista restrita, onde a ordem dos elementos define o significado do dado (Indexação Posicional). O conjunto de cartas de um jogador será uma lista que contém essas sublists.

Deve haver uma lista de gabarito que defina o nome e tipo de cada atributo das cartas:

Representação Visual da Implementação do gabarito (Super Trunfo de Carros)

```
# Nome
# Velocidade máxima (km/h)
# Tanque de combustível (litros)
# Ano criação (int)
```

Representação Visual da Implementação

```
# Exemplo de representação interna de uma carta:
# Índice 0: Nome (String)
# Índice 1: Atributo A (Float/Int)
# Índice 2: Atributo B (Float/Int)
# Índice 3: Atributo C (Float/Int)

carta_exemplo = ["Ferrari F40", 324, 4.1, 1350]

# Representação da mão do Jogador (Lista de Listas):
mao_jogador = [
    ["Ferrari F40", 324, 4.1, 1350], # Carta no topo
    ["Lamborghini", 330, 3.8, 1500], # Segunda carta
    ["Porsche 911", 310, 4.5, 1400] # Última carta
]
```

5. Fluxo de Execução Sugerido

1. **Criação do Baralho:** Criar uma lista principal contendo todas as cartas disponíveis.

2. **Embaralhamento:** Utilizar `random.shuffle()` na lista principal.
3. **Distribuição:** Dividir a lista principal em duas listas (`jogador_1` e `jogador_2`).
4. **Loop de Jogo:** Enquanto ambas as listas de jogadores tiverem comprimento maior que zero:
 - Exibir a carta do topo do jogador atual.
 - Solicitar a escolha do atributo (baseado no índice da lista).
 - Comparar com o índice correspondente na carta do adversário.
 - Mover as cartas para o vencedor ou tratar empate (monte de espera).
5. **Encerramento:** Declarar o vencedor final.

Bonus:

- Ler os detalhes e tipos de cartas de um arquivo texto
- Seu jogo pode ter vários tipos de super Trunfo, e os jogadores devem escolher com qual jogar
- Ranking de jogadores armazenado em arquivos